

Vel Tech Group of Institutions

Name: Gontu Yaswanth

Email: vtu33434@veltech.edu.in

Roll no: Vtu33434

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Competitive Coding II_L8

VTU_Week 9_CY

Attempt : 1

Total Mark : 40

Marks Obtained : 25

Section 1 : Coding

1. Problem Statement

Emily is working on arranging queens on a chessboard where one queen must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each other, with the condition that the queen at the specified position cannot be moved.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i, representing the row index of the fixed queen.

The third line contains an integer j, representing the column index of the fixed queen.

Output Format

If solutions exist: Print each valid board configuration, where "Q" represents a queen and "." represents an empty cell.

Each row of the board should be space-separated.

Each configuration should be followed by a blank line.

If no configuration is possible, print "No solution".

Refer to the sample output for the formatting specification.

Sample Test Case

Input: 4

0

1

Output: . Q . .

. . . Q

Q . . .

. . Q .

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <stdbool.h>
```

```
int N, fixedRow, fixedCol;  
int board[10]; // Stores column index for each row  
bool found = false;
```

```
// Check if placing a queen at (row, col) is safe
```

```
bool isSafe(int row, int col) {  
    for (int prevRow = 0; prevRow < row; ++prevRow) {  
        int prevCol = board[prevRow];  
        if (prevCol == col || abs(prevCol - col) == abs(prevRow - row)) {  
            return false;  
        }  
    }  
    return true;  
}
```

```
// Recursive backtracking function
```

```
void solve(int row) {  
    if (row == N) {  
        for (int i = 0; i < N; i++) {  
            for (int j = 0; j < N; j++) {  
                printf("%c%c", (board[i] == j ? 'Q' : '.'), (j == N - 1 ? '\0' : ' '));  
            }  
            printf("\n");  
        }  
        printf("\n");  
        found = true;  
        return;  
    }  
}
```

```
if (row == fixedRow) {  
    solve(row + 1);  
    return;  
}
```

```
for (int col = 0; col < N; col++) {  
    if (isSafe(row, col)) {  
        board[row] = col;  
        solve(row + 1);  
        board[row] = -1; // Reset  
    }  
}
```

```

    }
}

int main() {
    if (scanf("%d %d %d", &N, &fixedRow, &fixedCol) != 3) return 0;

    for (int i = 0; i < N; i++) board[i] = -1;
    board[fixedRow] = fixedCol;

    solve(0);

    if (!found) {
        printf("No solution\n");
    }

    return 0;
}

```

Status : Partially correct

Marks : 1/10

2. Problem Statement

Consider a rat placed at (0, 0) in a square matrix of order $N \times N$. It has to reach the destination at $(N - 1, N - 1)$. Find all possible paths that the rat can take to reach from the source to the destination. The directions in which the rat can move are 'U'(up), 'D'(down), 'L' (left), and 'R' (right). Value 0 at a cell in the matrix represents that it is blocked and the rat cannot move to it while value 1 at a cell in the matrix represents that the rat can travel through it. Return the list of paths in lexicographically increasing order.

Note: In a path, no cell can be visited more than once. If the source cell is 0, the rat cannot move to any other cell.

Example 1

Input:

$N = 4$

$m[i][j] =$

$\{ \{1, 0, 0, 0\},$

$\{1, 1, 0, 1\},$

$\{1, 1, 0, 0\},$

$\{0, 1, 1, 1\} \}$

Output:

DDRDRR DRDDRR

Explanation:

The rat can reach the destination at (3, 3) from (0, 0) by two paths - DRDDRR and DDRDRR, when printed in sorted order we get DDRDRR DRDDRR.

Example 2

Input:

N = 2

$m[i][j] =$

$\{ \{1, 0\},$

$\{1, 0\} \}$

Output:

-1

Explanation:

No path exists and the destination cell is blocked.

Input Format

The first line of input contains an integer n, representing the size of the maze.

The next n lines each contain n space-separated integers, representing the maze.

Output Format

The output consists of

If at least one path is found, print all valid paths as space-separated strings.

If no path is found, prints -1.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

1 0 0 0

1 1 0 1

1 1 0 0

0 1 1 1

Output: DDRDRR DRDDRR

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int N;
```

```
int maze[5][5];
```

```
int visited[5][5];
```

```
int found = 0;
```

```
void findPaths(int r, int c, char path[], int pathLen) {
```

```
    if (r == N - 1 && c == N - 1) {
```

```
        path[pathLen] = '\0';
```

```
        printf("%s ", path);
```

```
        found = 1;
```

```
        return;
```

```
    }
```

```
    visited[r][c] = 1;
```

```
    // D, L, R, U order for lexicographical requirement
```

```
    int dr[] = {1, 0, 0, -1};
```

```
    int dc[] = {0, -1, 1, 0};
```

```
    char dir[] = {'D', 'L', 'R', 'U'};
```

```

for (int i = 0; i < 4; i++) {
    int nr = r + dr[i];
    int nc = c + dc[i];

    if (nr >= 0 && nr < N && nc >= 0 && nc < N && maze[nr][nc] == 1 && !visited[nr]
[nc]) {
        path[pathLen] = dir[i];
        findPaths(nr, nc, path, pathLen + 1);
    }
}

visited[r][c] = 0; // Backtrack
}

int main() {
    if (scanf("%d", &N) != 1) return 0;

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &maze[i][j]);
            visited[i][j] = 0;
        }
    }

    if (maze[0][0] == 0 || maze[N - 1][N - 1] == 0) {
        printf("-1");
    } else {
        char path[26];
        findPaths(0, 0, path, 0);
        if (!found) printf("-1");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Given a partially filled 9x9 2D array, the goal is to assign digits (from 1 to 9)

to the empty cells so that every row, column, and sub-grid of size 3x3 contains exactly one instance of the digits from 1 to 9.

Write a program for the same.

Example

Input:

```
3 0 6 5 0 8 4 0 0
5 2 0 0 0 0 0 0 0
0 8 7 0 0 0 0 3 1
0 0 3 0 1 0 0 8 0
9 0 0 8 6 3 0 0 5
0 5 0 0 9 0 6 0 0
1 3 0 0 0 0 2 5 0
0 0 0 0 0 0 0 7 4
0 0 5 2 0 6 3 0 0
```

Output:

```
3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9
```

Explanation

To solve the given Sudoku puzzle, we apply the fundamental rules of Sudoku: each row, column, and 3x3 subgrid must contain the digits 1 through 9 without any repetition. Starting with the given partially filled grid, we systematically fill in the empty cells. For each empty cell, we check the numbers already present in the corresponding row, column, and 3x3 subgrid to determine which number can legally go in that position. This process involves logical deduction and elimination. In the case of the provided puzzle, the empty cells are filled one by one, ensuring that each row, column, and subgrid adheres to the Sudoku rules. For example, in Row 1, the missing numbers (1, 7, 9, 2) are placed in the available spots based on their placement in the columns and subgrids. This continues for each row, progressively narrowing down the possibilities for the empty cells until the entire grid is complete. The final solution, after applying this reasoning, looks like the output grid:

Input Format

The input consists of a 9x9 matrix in 9 rows and 9 columns separated by space.

Output Format

The output displays the 9x9 matrix with the updated values.

If the conditions are not satisfied, then display "No Solution exists".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 0 6 5 0 8 4 0 0

5 2 0 0 0 0 0 0 0

0 8 7 0 0 0 0 3 1

0 0 3 0 1 0 0 8 0

9 0 0 8 6 3 0 0 5

0 5 0 0 9 0 6 0 0

1 3 0 0 0 0 2 5 0

0 0 0 0 0 0 0 7 4

0 0 5 2 0 6 3 0 0

Output: 3 1 6 5 7 8 4 9 2

5 2 9 1 3 4 7 6 8

```
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9
```

Answer

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] board = new int[9][9];
```

```
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                board[i][j] = sc.nextInt();
            }
        }
```

```
        if (solveSudoku(board)) {
            printBoard(board);
        } else {
            System.out.println("No Solution exists");
        }
    }
```

```
    public static boolean solveSudoku(int[][] board) {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (board[row][col] == 0) {
                    for (int num = 1; num <= 9; num++) {
                        if (isSafe(board, row, col, num)) {
                            board[row][col] = num;
                            if (solveSudoku(board)) return true;
                            board[row][col] = 0; // Backtrack
                        }
                    }
                }
            }
        }
        return false;
```

```

    }
    }
    }
    return true;
}

public static boolean isSafe(int[][] board, int row, int col, int num) {
    for (int i = 0; i < 9; i++) {
        // Check row and column
        if (board[row][i] == num || board[i][col] == num) return false;
        // Check 3x3 subgrid
        if (board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] == num) return false;
    }
    return true;
}

public static void printBoard(int[][] board) {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            System.out.print(board[i][j] + (j == 8 ? "" : " "));
        }
        System.out.println();
    }
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Given an undirected graph and a number m , determine if the graph can be colored with at most m colors such that no two adjacent vertices of the graph are colored with the same color.

Note: Here, the coloring of a graph means the assignment of colors to all vertices.

Example

Input:

0 1 1 1

1 0 1 0

1 1 0 1

1 0 1 0

3

Output:

Solution Exists:

1 2 3 2

Explanation:

A minimum of 3 colors is required for the above graph.

Input Format

The input consists of four lines each containing four space-separated integers (1 if connected, 0 if not), representing the adjacency matrix of a graph where each element represents the connection between vertices.

The last line consists of an integer m, representing the number of colors available.

Output Format

If a solution exists:

1. The first line prints "Solution Exists:".
2. The second line prints the assigned colors for each vertex, separated by a space.

If no solution exists, the output prints "Solution does not exist".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0 1 1 1

1 0 1 0

1 1 0 1

1 0 1 0

3

Output: Solution Exists:

1 2 3 2

Answer

// You are using GCC

#include <stdio.h>

#include <stdbool.h>

int graph[4][4];

int colors[4];

int m;

// Function to check if color assignment is safe

bool isSafe(int v, int c) {

for (int i = 0; i < 4; i++) {

if (graph[v][i] && colors[i] == c) {

return false;

}

}

return true;

}

// Backtracking function to solve graph coloring

bool graphColoring(int v) {

if (v == 4) return true; // All vertices assigned

for (int c = 1; c <= m; c++) {

if (isSafe(v, c)) {

colors[v] = c;

if (graphColoring(v + 1)) return true;

colors[v] = 0; // Backtrack

}

}

return false;

}

```

int main() {
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            scanf("%d", &graph[i][j]);
        }
        colors[i] = 0;
    }
    scanf("%d", &m);

    if (graphColoring(0)) {
        printf("Solution Exists:\n");
        for (int i = 0; i < 4; i++) {
            printf("%d%c", colors[i], (i == 3 ? '\n' : ' '));
        }
    } else {
        printf("Solution does not exist");
    }

    return 0;
}

```

Status : Partially correct

Marks : 4/10